

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks: 25

SCENARIO-BASED QUESTIONS

Code of Conduct:

I V.Madhulika (192312620) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

1. With the help of a neat sketch, explain the different functional blocks present in a machine learning system, including data collection, preprocessing, feature extraction, model training, and evaluation. Illustrate the working of the system by providing suitable examples for both supervised and unsupervised learning approaches, clearly highlighting how labelled and unlabelled data are handled within the learning process.

Answer:

Functional Block Diagram:

Data Collection → Data Preprocessing → Feature Extraction → Model Training → Model Evaluation → Deployment/Prediction

Description of each block:

- Data Collection – gathering raw data from sensors, databases, surveys, or logs relevant to the problem.
- Preprocessing – cleaning the data (handling missing values, removing noise/outliers, normalization/scaling) so it is suitable for modeling.
- Feature Extraction – transforming raw attributes into meaningful, discriminative features (e.g., PCA, encoding categorical variables) that the model can learn from effectively.
- Model Training – fitting a chosen algorithm (e.g., neural network, decision tree) to the processed data by adjusting its parameters to minimize error.
- Evaluation – testing the trained model on unseen data using metrics such as accuracy, precision, and recall to judge performance before deployment.

Supervised Learning Example:

Email spam detection – each email in the training data is labelled 'spam' or 'not spam'. The model learns a mapping from input features (word frequency, sender, links) to the known label, and its predictions are directly compared against these labels during training and evaluation.

Unsupervised Learning Example:

Customer segmentation – customer purchase data has no predefined labels. A clustering algorithm (e.g., K-Means) groups customers with similar buying patterns into clusters purely from the structure of the unlabelled data, which marketers can later interpret and act upon.

In summary, labelled data drives supervised learning by providing a direct target for the model to match, whereas unlabelled data in unsupervised learning is handled by discovering inherent patterns, groupings, or structure without any predefined output.

2. Design a machine learning system for disaster management to predict events such as floods and earthquakes using both historical and real-time data sources. Explain the selection of an appropriate model, including whether a neural network or any other algorithm is suitable for this task, along with justification. Describe the data preprocessing steps and feature selection techniques required to improve model performance. Further, explain the complete process of training, validating, and evaluating the model, and discuss how such a system can assist authorities in early warning, risk assessment, and effective emergency response.

Answer:

System Design:

Inputs: historical records of past floods/earthquakes, real-time sensor feeds (rainfall gauges, river levels, seismographs), satellite imagery, and weather forecasts.

Model Selection:

A Recurrent Neural Network (LSTM) or a hybrid CNN-LSTM is well suited since disaster indicators are sequential/time-series in nature (rainfall over hours/days, seismic tremor patterns) and LSTMs capture long-term temporal dependencies better than simple feed-forward networks. For smaller structured datasets, ensemble methods such as Random Forest or Gradient Boosting can also be justified for their robustness and interpretability.

Preprocessing & Feature Selection:

- Handle missing sensor readings via interpolation; remove outlier/faulty sensor spikes.
- Normalize features (rainfall, water level, magnitude) to a common scale.
- Select features using correlation analysis and domain knowledge — e.g., rainfall intensity, soil saturation, and river discharge for floods; seismic frequency and depth for earthquakes.

Training, Validation & Evaluation:

The dataset is split into training, validation, and test sets (respecting time order to avoid data leakage). The model is trained using backpropagation-through-time (for LSTM) while monitoring validation loss to apply early stopping. Performance is evaluated using recall (critical to avoid missed disasters), precision, and F1-score, since missed warnings are far costlier than false alarms.

Impact on disaster management:

Such a system enables early warning by flagging rising risk levels before an event occurs, supports risk assessment by quantifying probability and expected severity for specific regions, and helps authorities coordinate evacuation, resource allocation, and emergency response well in advance, ultimately reducing loss of life and property.

3. In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model.

Answer:

Forward Pass:

Input values are multiplied by their respective weights, summed with the bias at each neuron, and passed through an activation function (e.g., sigmoid) to produce the neuron's output. This continues layer by layer until the final output layer produces a predicted probability (e.g., 0/1 class).

Error Calculation:

The error is computed by comparing the predicted output to the actual target using a loss function, e.g., $E = \frac{1}{2}(\text{target} - \text{output})^2$ for a single sample, or cross-entropy loss for classification.

Backward Propagation:

The error is propagated backward from the output layer to the hidden layers using the chain rule of calculus. For each weight, the gradient of the error with respect to that weight is calculated, and the weight is updated as: $w(\text{new}) = w(\text{old}) - \eta \times (\partial E / \partial w)$, where η is the learning rate. This process repeats for every training example across multiple epochs until the error is minimized.

Influence of Learning Rate:

A learning rate that is too high can cause the weight updates to overshoot the minimum, leading to oscillation or divergence, while a rate that is too low results in very slow convergence and can get stuck in flat regions. A well-tuned or adaptive learning rate (e.g., Adam optimizer) balances speed and stability.

Influence of Activation Functions:

Sigmoid/tanh functions can cause vanishing gradients in deep networks, slowing learning in early layers, whereas ReLU and its variants mitigate this and speed up convergence. The choice of activation function directly affects how well gradients flow backward and, consequently, the final accuracy and training stability of the model.

4. In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution.

Answer:

Example	Speed	Accuracy	Fitness	Suitable
1	High	Low	40	No
2	Medium	Medium	60	Yes
3	High	Medium	75	Yes
4	Low	High	65	Yes
5	Medium	High	80	Yes

Selection:

Fitness-proportionate selection favors individuals with the highest fitness. Example 5 (fitness 80) and Example 3 (fitness 75) are the fittest and are selected as parents for reproduction, while low-fitness Example 1 (40) is unlikely to be chosen again.

Crossover:

Combining the Speed attribute of Example 3 (High) with the Accuracy attribute of Example 5 (High) produces a new offspring chromosome (High Speed, High Accuracy). Since both contributing attributes individually correlate with high fitness, this offspring is expected to have a fitness at or above 80, likely marked 'Yes'.

Mutation:

Randomly altering one attribute, e.g., changing Example 2's Accuracy from Medium to High, explores a region of the hypothesis space not directly reachable by crossover of the current population, helping to discover better combinations and avoid premature convergence around Examples 3 and 5 only.

Generational Improvement & Optimal Solution:

Across generations, low-fitness combinations (like High Speed/Low Accuracy) are gradually eliminated while High Speed–High Accuracy combinations dominate. The best optimal solution that emerges is the (High Speed, High Accuracy) candidate — combining the strongest attributes of Examples 3 and 5 — yielding the highest fitness and a 'Suitable = Yes' classification.

5. In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and

Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs.

Answer:

Example	Age	BP	Glucose	Lifestyle Risk	Actual	Predicted
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Training with backpropagation:

Each patient record's attributes (Age, BP, Glucose, Lifestyle Risk) are encoded numerically and fed into the input layer. The network computes a forward pass to produce a predicted probability of disease, compares it to the actual label using a loss function, and propagates the error backward to update the weights via gradient descent. This is repeated over many epochs across the training set until the network reliably maps attribute patterns to the correct class.

Confusion Matrix Values (from the table, positive class = 'Yes'):

Example 1: Actual Yes, Predicted Yes → TP. Example 2: Actual Yes, Predicted No → FN. Example 3: Actual No, Predicted No → TN. Example 4: Actual Yes, Predicted Yes → TP. Example 5: Actual No, Predicted Yes → FP.

TP = 2, TN = 1, FP = 1, FN = 1, Total = 5

Metric	Formula	Calculation	Result
Accuracy	$(TP+TN)/\text{Total}$	$(2+1)/5$	60%
Precision	$TP/(TP+FP)$	$2/3$	66.67%
Recall	$TP/(TP+FN)$	$2/3$	66.67%
Specificity	$TN/(TN+FP)$	$1/2$	50%

On this small sample, the model correctly identifies most cases but has one false negative (a missed disease case) and one false positive, indicating room for improvement, especially in recall for a medical context. Proper preprocessing (handling missing/inconsistent readings, scaling Age/Glucose values) and feature selection (identifying which attributes — e.g., Glucose and Lifestyle Risk — are most predictive) can reduce noise and redundancy, helping the network focus on the most informative signals and thereby improving both accuracy and reliability.

6. With the help of a neat diagram, explain the architecture of a decision tree learning system. Describe how features are selected using measures such as entropy and information gain. Illustrate the process of tree construction and pruning using a small example dataset, and explain how overfitting can be controlled in decision tree models.

Answer:

Architecture:

A decision tree consists of a root node (the entire dataset), internal decision nodes (each testing an attribute), branches (representing possible attribute values/outcomes), and leaf nodes (final class labels). Diagrammatically: Root → [Attribute Test] → Branches → further Attribute Tests → Leaf Nodes (class predictions).

Feature Selection using Entropy & Information Gain:

Entropy measures impurity/disorder in a set: $E(S) = -\sum p_i \log_2(p_i)$, where p_i is the proportion of class i . Information Gain for an attribute A is $IG(S,A) = E(S) - \sum (|S_v|/|S|) \cdot E(S_v)$, the reduction in entropy achieved by splitting on A . The attribute with the highest information gain is chosen at each node, since it best separates the classes.

Tree Construction Example:

For a small weather dataset (Outlook, Humidity $\rightarrow$ Play Yes/No), the algorithm computes entropy of the full dataset, then information gain for each candidate attribute (Outlook, Humidity, Wind). Suppose 'Outlook' yields the highest gain; it becomes the root split. This process recurses on each branch (e.g., further splitting 'Sunny' branch by Humidity) until nodes are pure or a stopping criterion is reached.

Pruning:

Pre-pruning stops tree growth early (e.g., limiting depth or minimum samples per leaf), while post-pruning grows a full tree and then removes branches that provide little predictive value on a validation set, replacing them with a leaf representing the majority class.

Controlling Overfitting:

- Pruning (pre- or post-) to avoid overly deep, highly specific trees.
- Setting a minimum number of samples required to split a node or form a leaf.
- Using cross-validation to select the optimal tree depth/complexity.
- Ensemble methods such as Random Forests, which average many trees to reduce variance.

7. Design a machine learning system for smart traffic management to predict traffic congestion using data such as vehicle density, time of day, weather conditions, and road type. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time decision-making.

Answer:

System Design & Data Sources:

Inputs include real-time vehicle density from traffic cameras/sensors, time-of-day and day-of-week patterns, weather data (rain, fog reducing visibility/speed), and static road-type information (highway, junction, residential).

Algorithm Choice & Justification:

Since congestion evolves over time and depends on recent history, a Recurrent Neural Network (LSTM) or Gradient Boosted Trees (e.g., XGBoost) on engineered time-window features are both suitable. LSTM is justified when abundant sequential sensor data is available and temporal dependencies (rush-hour build-up) are important; Gradient Boosting is a strong, faster-to-train alternative for structured tabular features when data volume is moderate.

Preprocessing & Feature Engineering:

- Clean/normalize sensor readings and impute missing values from adjacent time windows.
- Encode time-of-day and day-of-week cyclically (sine/cosine transforms) so the model understands their periodic nature.
- Engineer rolling averages of vehicle density and lag features (density 15/30 minutes ago) to capture trend.
- One-hot encode categorical features like road type and weather condition.

Training, Validation & Deployment:

The model is trained on historical traffic logs, validated on a held-out time period (not randomly shuffled, to respect temporal order), and tuned using metrics like RMSE for congestion-level prediction or F1-score if framed as a congestion/no-congestion classification. Once validated, the model is deployed on edge servers or a central traffic control system, ingesting live sensor streams to output real-time congestion forecasts that drive adaptive traffic-signal timing and route-advisory systems for commuters.

8. In the context of Convolutional Neural Networks (CNNs), explain the different layers involved, including convolutional, pooling, and fully connected layers. Describe how feature maps are generated and how backpropagation updates weights in CNNs. Illustrate the process with an example of image classification and discuss factors affecting model performance.

Answer:

CNN Layers:

- Convolutional Layer – slides small learnable filters (kernels) across the input image, computing dot products to detect local patterns such as edges, textures, and shapes; each filter produces a feature map.
- Activation (ReLU) – introduces non-linearity by zeroing out negative values in the feature maps.
- Pooling Layer – down-samples feature maps (e.g., max-pooling takes the largest value in each region), reducing spatial dimensions and computational cost while retaining the most important features.
- Fully Connected Layer – flattens the high-level feature maps and connects every neuron to every neuron in the next layer to perform final classification into output classes.

Feature Map Generation:

Each filter in a convolutional layer is convolved across the width and height of the input, producing one 2D feature map per filter; stacking multiple filters produces a volume of feature maps that captures different aspects (edges, corners, textures) of the input.

Backpropagation in CNNs:

The classification error at the output is propagated backward through the fully connected layers and then through the pooling and convolutional layers using the chain rule, computing gradients with respect to each filter's weights. Filters are then updated via gradient descent, gradually learning to detect features most relevant for reducing classification error.

Example — Image Classification:

For classifying handwritten digits, early convolutional layers learn to detect simple strokes and edges, deeper layers combine these into shapes like loops and curves, and the fully connected layer maps these high-level features to digit classes (0–9).

Factors Affecting Performance:

- Depth/number of layers and filters (model capacity).
- Quality and quantity of training data, and use of data augmentation.
- Choice of activation functions and proper weight initialization.
- Regularization techniques (dropout, batch normalization) to prevent overfitting.

9. Consider the application of Genetic Algorithms for feature selection in a machine learning problem. Explain how candidate feature subsets are represented as chromosomes and how fitness is evaluated. Describe the role of selection, crossover, and mutation in evolving better feature subsets, and discuss how this improves model accuracy and reduces complexity.

Answer:

Chromosome Representation:

Each chromosome is typically a binary string of length equal to the total number of available features, where a '1' indicates the feature is included in the subset and '0' indicates it is excluded. For example, for 6 features, the chromosome 101010 selects features 1, 3, and 5.

Fitness Evaluation:

A model (e.g., a classifier) is trained using only the features indicated by the chromosome, and its performance (accuracy, F1-score) on a validation set — often combined with a penalty for the number of features used — serves as the fitness value, rewarding subsets that achieve high accuracy with fewer features.

Selection:

Chromosomes (feature subsets) with higher fitness are more likely to be chosen as parents, ensuring that promising feature combinations propagate to the next generation.

Crossover:

Two parent chromosomes exchange segments of their bit strings (e.g., single-point or uniform crossover) to create offspring subsets that combine useful features from both parents.

Mutation:

Randomly flips a bit (turning a feature on/off) in an offspring, allowing the algorithm to explore feature combinations not present in either parent and avoid premature convergence to a locally optimal subset.

Impact on Model Accuracy & Complexity:

By repeatedly selecting, combining, and mutating feature subsets over generations, the Genetic Algorithm converges toward a subset that removes irrelevant or redundant features. This reduces model complexity, training time, and the risk of overfitting, while often improving accuracy by eliminating noisy features that would otherwise mislead the learning algorithm.

10. In the context of classification problems, explain how different evaluation metrics such as accuracy, precision, recall, F1-score, and ROC curve are used to assess model performance. Using a sample confusion matrix, demonstrate the calculation of these metrics and discuss their importance in applications such as medical diagnosis and fraud detection.

Answer:

Sample Confusion Matrix:

Consider a classifier evaluated on 200 samples: TP = 80, TN = 90, FP = 15, FN = 15.

Metric	Formula	Calculation	Result
Accuracy	$(TP+TN)/\text{Total}$	$(80+90)/200$	85%
Precision	$TP/(TP+FP)$	$80/95$	84.21%
Recall (Sensitivity)	$TP/(TP+FN)$	$80/95$	84.21%
F1-Score	$2 \cdot (P \cdot R)/(P+R)$	$2(0.8421 \times 0.8421)/(1.6842)$	84.21%

The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate against the False Positive Rate at various classification thresholds; the Area Under the Curve (AUC) summarizes overall discriminative ability across all thresholds, with a value closer to 1 indicating a better classifier.

Importance in real-world applications:

- Medical diagnosis – recall (sensitivity) is prioritized since missing a true disease case (false negative) can be life-threatening, even at the cost of some extra false alarms.
- Fraud detection – precision is often prioritized since flagging too many legitimate transactions as fraud (false positives) damages customer trust, while F1-score balances both concerns.
- Accuracy alone can be misleading on imbalanced datasets (e.g., rare disease or rare fraud cases), so precision, recall, F1-score, and ROC-AUC together give a much more complete and trustworthy picture of model performance.

11. With the help of a neat diagram, explain the architecture of an Artificial Neural Network (ANN). Describe the roles of input, hidden, and output layers, and explain how weights and biases are used in computation. Illustrate how forward propagation works with a simple example, and discuss how overfitting can be controlled using techniques such as dropout and regularization.

Answer:

ANN Architecture:

Input Layer → Hidden Layer(s) → Output Layer, with every neuron in one layer connected to every neuron in the next via weighted edges, plus a bias term per neuron.

- Input Layer – receives the raw feature values (e.g., pixel intensities, sensor readings) and passes them forward unchanged.
- Hidden Layer(s) – perform weighted summation of inputs plus bias, then apply a non-linear activation function (ReLU, sigmoid) to learn complex, non-linear relationships in the data.
- Output Layer – produces the final prediction (e.g., a class probability via softmax/sigmoid).

Weights and Biases:

Each connection has a weight that scales the signal passing through it, and each neuron has a bias that shifts the weighted sum before activation, allowing the network to fit patterns that don't necessarily pass through the origin. Together, weights and biases are the learnable parameters adjusted during training.

Forward Propagation Example:

For inputs $x_1=1$, $x_2=0.5$ with weights $w_1=0.4$, $w_2=0.6$ and bias $=0.1$: $\text{net} = (0.4 \times 1) + (0.6 \times 0.5) + 0.1 = 0.8$. Passing this through a sigmoid activation gives output ≈ 0.69 , which becomes the input to the next layer (or the final prediction if this is the output layer).

Controlling Overfitting:

- Dropout – randomly deactivates a fraction of neurons during each training pass so the network cannot over-rely on specific neurons, forcing more robust, redundant representations.
- Regularization (L1/L2) – adds a penalty term proportional to the magnitude of the weights to the loss function, discouraging overly large weights and overly complex decision boundaries.

12. Design a machine learning system for weather forecasting to predict rainfall using features such as temperature, humidity, wind speed, and atmospheric pressure. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time predictions.

Answer:

System Design:

Historical and real-time meteorological data (temperature, humidity, wind speed, atmospheric pressure, cloud cover) collected from weather stations and satellites form the input to the system, with the target being rainfall amount (regression) or rain/no-rain (classification).

Algorithm Choice & Justification:

An LSTM-based recurrent network is well suited because weather is inherently a time-series process where current conditions strongly depend on recent trends (e.g., falling pressure preceding rain). Alternatively, ensemble models like Random Forest or Gradient Boosting are strong choices for shorter-horizon, feature-based forecasting due to their robustness to noisy sensor data and ability to model non-linear interactions between variables.

Preprocessing & Feature Engineering:

- Impute missing sensor readings and remove faulty/outlier measurements.
- Normalize/standardize temperature, humidity, and pressure to comparable scales.
- Engineer features such as pressure trend (rate of change over past hours), humidity–temperature interaction (dew point), and seasonal/monthly indicators.

Training, Validation & Deployment:

The model is trained on multi-year historical data, validated on a separate time period (to mimic real forecasting conditions), and evaluated using RMSE/MAE for rainfall amount or precision-recall for rain/no-rain classification. Once validated, it is deployed to continuously ingest live sensor and satellite feeds, producing rolling short-term rainfall forecasts that support agriculture planning, flood warnings, and daily public advisories.

13. In the context of Recurrent Neural Networks (RNNs), explain the architecture and how they handle sequential data. Describe the concept of hidden states and how information is passed through time steps. Illustrate with an example such as text prediction or time-series forecasting, and discuss challenges like vanishing gradients and possible solutions.

Answer:

RNN Architecture:

Unlike feed-forward networks, an RNN has a recurrent connection that feeds the output of a hidden layer back into itself at the next time step, allowing the network to maintain a form of memory of prior inputs as it processes a sequence one element at a time.

Hidden States:

At each time step t , the hidden state $h(t)$ is computed as a function of the current input $x(t)$ and the previous hidden state $h(t-1)$: $h(t) = f(W_x \cdot x(t) + W_h \cdot h(t-1) + b)$. This hidden state acts as a compressed summary of everything the network has seen in the sequence so far, and is passed forward to influence the computation at the next time step.

Example — Text Prediction:

When predicting the next word in 'The sky is ___', the RNN processes 'The', 'sky', 'is' sequentially, updating its hidden state at each word so that by the time it reaches the blank, the hidden state encodes the relevant context, allowing it to predict a plausible next word such as 'blue'. Similarly, in time-series forecasting, the hidden state accumulates trend/seasonality information from past readings to predict the next value.

Vanishing Gradient Challenge:

During backpropagation-through-time, gradients are repeatedly multiplied across many time steps; when these values are small, the gradient shrinks exponentially, so the network struggles to learn dependencies that span many steps back — it effectively 'forgets' long-term context.

Solutions:

- LSTM/GRU units – use gating mechanisms (input, forget, output gates) that allow gradients to flow more directly across time steps, preserving long-term dependencies.
- Gradient clipping – caps the gradient magnitude during training to prevent instability.
- Attention mechanisms – let the model directly reference relevant earlier time steps rather than relying solely on a compressed hidden state.

14. Consider the use of Genetic Algorithms for optimization problems such as scheduling tasks in a system. Explain how solutions are encoded as chromosomes and how the fitness function is defined. Describe the processes of selection, crossover, and mutation, and discuss how these operations help in finding near-optimal solutions efficiently.

Answer:

Chromosome Encoding:

For a task-scheduling problem, a chromosome can be represented as a permutation or sequence of task IDs (e.g., [T3, T1, T4, T2]) indicating the order in which tasks are executed, or as an array mapping each task to an assigned machine/time-slot.

Fitness Function:

Fitness is typically defined as the inverse of total completion time (makespan), resource idle time, or the number of scheduling conflicts/deadline violations — schedules with shorter makespan and fewer conflicts receive higher fitness scores.

Selection:

Schedules with the best fitness (shortest completion time, fewest conflicts) are more likely to be chosen as parents for the next generation, e.g., via roulette-wheel or tournament selection.

Crossover:

Two parent schedules exchange sub-sequences of tasks (using techniques such as order crossover, which preserves the validity of the permutation) to produce new candidate schedules that combine good partial orderings from both parents.

Mutation:

Randomly swaps the positions of two tasks in a schedule or reassigns a task to a different machine, introducing variation that helps the search escape a locally optimal ordering.

Efficiency in finding near-optimal solutions:

By iteratively selecting fitter schedules, recombining their good sub-sequences through crossover, and injecting variation through mutation, the Genetic Algorithm explores a huge combinatorial search space far more efficiently than exhaustive search, converging over generations toward schedules with low makespan and minimal conflicts without needing to evaluate every possible ordering.

15. In the context of classification models, explain how performance evaluation is carried out using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Using a sample confusion matrix, demonstrate how each metric is calculated and discuss their importance in real-world applications such as spam detection and disease diagnosis.

Answer:

Sample Confusion Matrix:

Consider a spam classifier tested on 300 emails: TP = 100 (spam correctly flagged), TN = 160 (non-spam correctly passed), FP = 20 (non-spam wrongly flagged), FN = 20 (spam missed).

Metric	Formula	Calculation	Result
Accuracy	$(TP+TN)/Total$	$(100+160)/300$	86.67%
Precision	$TP/(TP+FP)$	$100/120$	83.33%
Recall (Sensitivity)	$TP/(TP+FN)$	$100/120$	83.33%
F1-Score	$2 \cdot (P \cdot R)/(P+R)$	$2(0.8333 \times 0.8333)/(1.6667)$	83.33%

ROC-AUC is obtained by plotting True Positive Rate versus False Positive Rate across all possible decision thresholds and measuring the area under this curve; a value near 1.0 indicates the classifier separates the two classes very well across thresholds, not just at one fixed cutoff.

Importance in real-world applications:

- Spam detection – precision matters a great deal, since a high false-positive rate means important legitimate emails get lost in the spam folder, frustrating users.
- Disease diagnosis – recall matters most, since a false negative means a real illness goes undetected and untreated, which can be far more dangerous than a false alarm.
- Accuracy alone can hide poor performance on the minority class in imbalanced data, so precision, recall, F1-score, and ROC-AUC are used together to give a balanced, application-appropriate view of a classifier's real-world reliability.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand